Week 5: Introduction to AI and LLMs

Python 200 | Mentor Session

What is a Large Language Model?

A neural network trained on enormous amounts of text — books, web pages, code — to predict what word comes next.

Do that at massive scale, and something remarkable emerges: a model that can answer questions, write code, summarize documents, translate languages, and more.

You don't need to train one. You interact with them through APIs.

> **Discussion:** What LLMs have you used? What surprised you about what they could — or couldn't — do?

LLMs You Already Know

- **ChatGPT** (OpenAI) — GPT-4o and family
- **Claude** (Anthropic) — the model behind this course
- **Gemini** (Google)
- **Llama** (Meta) — open weights, runs locally

All of these are neural networks. Different training data, different design choices — but the same fundamental idea.

Tokens: What They Are

LLMs don't read words — they read **tokens**: chunks of roughly 3–4 characters each.

Text	Tokens
`"hello"`	1 token
`"unbelievable"`	3 tokens
`"I love pizza"`	3 tokens

Tokenization happens before any text enters the model.

Why Tokens Matter

- **Cost** — most APIs charge per token
- **Context limit** — models have a maximum number of tokens they can process at once (e.g. GPT-4o: 128,000)
- **Non-English text and code** use more tokens per character than plain English

> **Discussion:** You're building a pipeline that processes thousands of documents with an LLM. Why would token count matter beyond just response speed?

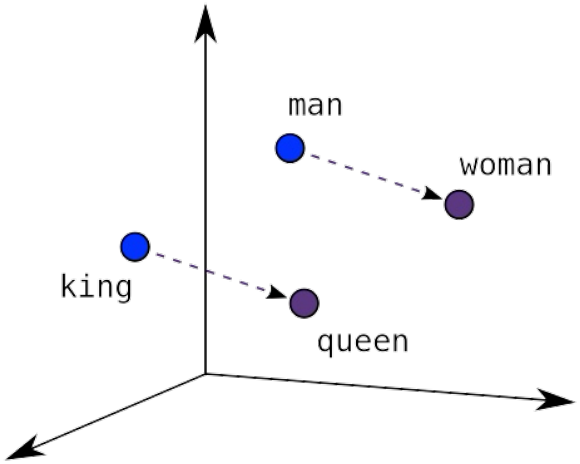
Embeddings: Text as Numbers

An **embedding** is a list of numbers that represents the *meaning* of a piece of text.

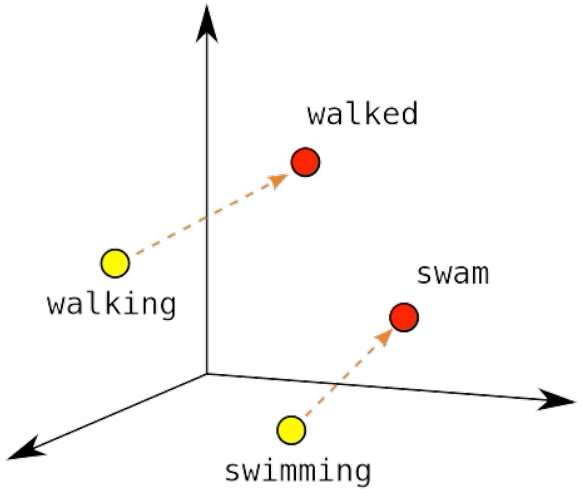
- `"king"` → `[0.23, -0.81, 0.54, ...]` (hundreds of dimensions)
- Similar meanings → numbers that are close together in space
- `"car"` and `"automobile"` land near each other

The model learned these positions from patterns in training data — words used in similar contexts get similar embeddings.

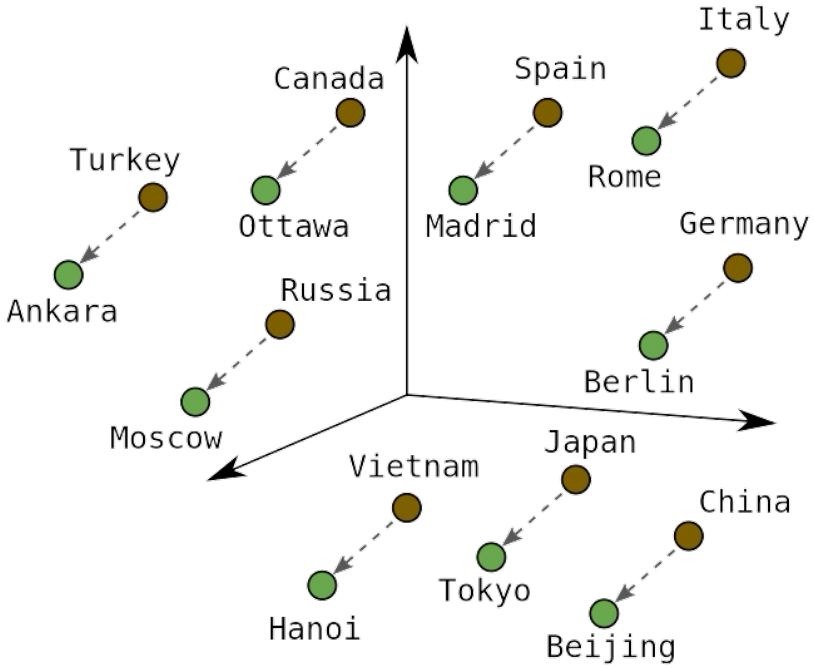
See next slide for diagram →



Male-Female



Verb Tense



Country-Capital

How LLMs Generate Text

1. You send a prompt
2. The model assigns a probability to every possible next token
3. It samples from that distribution and adds the token to the output
4. Repeat until done

This is **next-token prediction** — the model doesn't "know" answers, it predicts what a likely answer looks like.

Temperature

Temperature controls how random the sampling is:

- **Low (0.1–0.3)** → focused, predictable, consistent
- **High (0.8–1.0)** → creative, varied, sometimes surprising

This is why LLMs can confidently say things that are wrong — high probability ≠ true.

> **Discussion:** Given that LLMs predict what "sounds right" rather than look things up — what kinds of tasks are they well-suited for? What could go badly wrong?

The Chat Completions API

Every message has a **role** and **content**:

- `system` — sets the model's behavior ("You are a helpful assistant.")
- `user` — your prompt
- `assistant` — the model's previous response (for multi-turn conversations)

The model reads all messages in order and generates the next `assistant` response.

Chat Completions: The Code

```
from openai import OpenAI
client = OpenAI()

response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user",   "content": "What is a token?"}
    ]
)

print(response.choices[0].message.content)
```

Kaggle Demo: Chat Completions *(Lesson 2)*

Open: Lesson 2 → "Open in Kaggle" button

Focus on	Skim for now
Message structure and roles	The moderations endpoint
Reading the response object	Error handling details
Changing temperature, watching output vary	

Try changing the system message to "You are a pirate" — how does the response change?

> **Discussion:** What's the difference between the system message and the user message? Why would you want to keep them separate?

The Memory Problem

Every API call is **stateless** — the model has no memory of previous requests.

Each call is a fresh start. If you ask "what did I just say?" it has no idea, even if you've been talking for an hour.

This isn't a bug — it's how the API is designed. Every production chatbot has to solve this the same way.

> **Discussion:** If each API call is completely independent, how do you think existing chatbots like ChatGPT "remember" what you said earlier in the conversation?

Adding Memory: Pass the History

The solution: **send the entire conversation with every call**.

```
messages = [{"role": "system", "content": "You are a helpful assistant."}]

# Each turn:
messages.append({"role": "user",      "content": user_input})
response = client.chat.completions.create(model="gpt-4o", messages=messages)
messages.append({"role": "assistant", "content": response.choices[0].message.content})
```

The model "remembers" because you're telling it what was said. Your code manages the list — the server stores nothing.

Kaggle Demo: Chatbots *(Lesson 3)*

Open: Lesson 3 → "Open in Kaggle" button

Focus on	Skim for now
How the message list grows with each turn	Streaming responses
That the model "remembers" because of the history you pass	Token count management
What happens if you remove old messages	

> **Discussion:** You're building a chatbot serving thousands of users with hour-long sessions. What happens to the token count over time? How would you handle it?

Prompt Engineering: The Core Techniques

Zero-shot — just the instruction, no examples

> "Classify this review as positive or negative."

One-shot — one example of what you want

> "Here's an example: [example]. Now do the same for: [input]."

Few-shot — two to five examples

> Best when consistency of format or style matters

Chain-of-Thought Prompting

Ask the model to reason step by step before giving an answer.

```
"Think through this carefully before responding."
```

Or just add: `"Let's think step by step."`

This dramatically improves accuracy on reasoning, math, and multi-step tasks. The model "thinks out loud" and catches its own errors.

> **Discussion:** Why would "thinking step by step" help a model get better answers? What might it be doing that improves accuracy?

Kaggle Demo: Prompt Engineering *(Lesson 4)*

Open: Lesson 4 → "Open in Kaggle" button

Focus on	Skim for now
Comparing zero-shot vs. few-shot outputs side by side	JSON mode / structured output
How adding examples changes response format	Advanced system message patterns
Chain-of-thought on a reasoning task	

Pick one task and write the worst possible prompt, then iterate until it works well.

> **Discussion:** What's one prompt you've tried that didn't work the way you expected? What did you change to fix it?

AI Ethics: Questions Worth Taking Seriously

- **Bias** — models trained on internet data inherit human biases and can amplify them
- **Misinformation** — LLMs generate plausible-sounding text, not verified facts; confident falsehoods at scale
- **Energy use** — training and running large models has real, contested environmental costs
- **Labor displacement** — automation of knowledge work affects real livelihoods, especially in translation, writing, and support

These are active debates — not footnotes.

> **Discussion:** Some argue we should slow down or stop building more powerful AI. Others say benefits outweigh risks. Is there a position you find compelling?

Week 5 Wrap-Up

- **LLMs** — trained to predict text at massive scale; powerful but not infallible
- **Tokens & embeddings** — how models turn text into numbers; the foundation of everything else
- **Chat Completions API** — roles + messages → response; the interface you'll use in every LLM pipeline
- **Memory** — LLMs are stateless; you manage conversation history yourself
- **Prompt engineering** — zero-shot, few-shot, chain-of-thought; the craft of getting reliable outputs
- **Ethics** — real costs and risks worth understanding

Takeaway: Understanding how LLMs actually work — tokens, context windows, statelessness — separates engineers who use them well from those who get surprised by them.